SARA (सार)

Knowledge Distillation — Theory, Techniques & KD-SPAR

Author	Ashutosh Sinha
Email	ajsinha@gmail.com
Domain	Machine Learning · Model Compression · Deep Learning
Version	4.0 (April 2025)
Audience	ML Engineers · Applied Researchers · AI Architects

© 2025 Ashutosh Sinha (ajsinha@gmail.com). All rights reserved. Reproduction or redistribution without written permission is prohibited.

Abstract

Knowledge distillation (KD) has evolved from a model-compression curiosity into a foundational engineering pattern for the modern AI stack. This document provides a comprehensive technical reference spanning eight parts: theoretical foundations of soft-target training; a taxonomy of five distillation families; end-to-end implementations in PyTorch, Lightning, and Hugging Face; benchmarks across vision, language, and speech; and a full treatment of KD applied to production RAG systems where the generative model is not fixed but routinely replaced.

The document's primary novel contribution is KD-SPAR — Knowledge Distillation via Student Prompt Auto-Rewriting — a paradigm in which the student model itself diagnoses where its outputs diverge from the teacher and proposes targeted amendments to its own system prompt. Three variants are developed in full: Multi-Teacher KD-SPAR, which satisfies constraints from N specialist teachers simultaneously without regressing any; Adversarial KD-SPAR, which mines hard examples and generates adversarial queries to build prompt robustness beyond the common-query distribution; and Federated KD-SPAR, which enables distributed client sites to jointly optimise a global prompt without sharing any raw query or response data — only proposed instruction strings cross site boundaries. A fifth variant, MetaKDSPAR, integrates metaprompting: four specialist diagnostic perspectives (citation, calibration, completeness, format) independently analyse failures, a conductor synthesises diagnoses, and each specialist proposes domain-specific fixes.

The document concludes with a rigorous ablation experiment design that directly tests the self-knowledge hypothesis: four controlled conditions (student self-proposed, teacher externally-proposed, random, and no-tuning baseline) are compared on identical evaluation queries. A complete local-model backend using Ollama is provided, enabling the experiment to run on a single GPU workstation (System76 OryxPro / Pop!_OS) with no API cost, no rate limits, and full reproducibility. Recommended model pairs: llama3.1:8b → llama3.2:3b (same-family controlled) and qwen2.5:7b → llama3.2:3b (cross-family generalisation test). The document concludes with an assessment of the novelty of the KD-SPAR paradigm relative to the existing literature on prompt optimisation, self-refinement, and constitutional AI.

Document Structure

Part	Sections	Focus
I — Foundations	1–3b	Theory, taxonomy, literature survey
II — Implementation	4–5	Code, advanced techniques
III — Applications	6–7	Real-world cases, benchmarks
IV — RAG & Model Migration	8–9	Pipeline, migration, prompt KD
V — KD-SPAR Variants	10–13b	Base, Multi-Teacher, Adversarial, Federated, MetaKDSPAR
VI — Practitioner Guide	14–17	Hyperparams, eval, best practices, frameworks
VII — Frontier	18–18e	Future directions, Related Work, Algorithm 1, Limitations, Conclusion

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

Part	Sections	Focus
VIII — Novelty & Research	19–21	Novelty assessment, ablation results, local model backend

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

PART I **Foundations** *Theory · Soft Targets · Taxonomy*

1. Introduction to Knowledge Distillation

Knowledge distillation (KD) is a model compression and transfer-learning technique in which a smaller, faster student network is trained to replicate the behaviour of a larger, more capable teacher network. Originally formalised by Hinton, Vinyals & Dean (2015), KD has become one of the most widely deployed techniques for deploying state-of-the-art AI into resource-constrained environments.

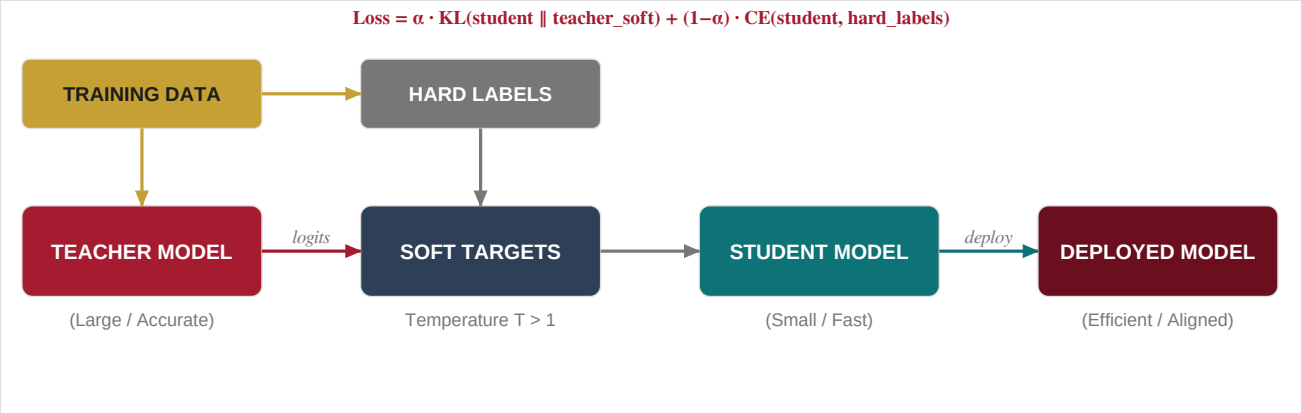

Figure 1.1 — KD flow: the teacher produces soft targets consumed alongside hard labels; Loss = α·KL(soft) + (1−α)·CE(hard).

Why It Matters

Large foundation models require tens of gigabytes of VRAM and hundreds of watts. KD unlocks their capability inside laptops, phones, and microcontrollers without sacrificing the reasoning depth that makes them useful.

1.1 Historical Background

Model compression was pioneered by Bucilua et al. (2006). Hinton et al. (2015) generalised this with the temperature-scaled softmax — the canonical KD formulation used today. The field has since branched into feature-level, attention-level, relation-level, and data-free paradigms.

1.2 Core Intuition — Dark Knowledge

A teacher trained on hard labels implicitly encodes rich inter-class similarity — knowing that cat is more similar to dog than to airplane. This structure, encoded in the soft output probabilities, is Hinton's dark knowledge. The student learns these rather than hard labels alone.

© 2025 Ashutosh Sinha. All rights reserved.

Analogy

A master craftsperson narrating reasoning aloud as an apprentice observes — the apprentice learns not just the final product but the decision-making process embedded in every intermediate judgement.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

2. Theoretical Foundations

2.1 Temperature Scaling

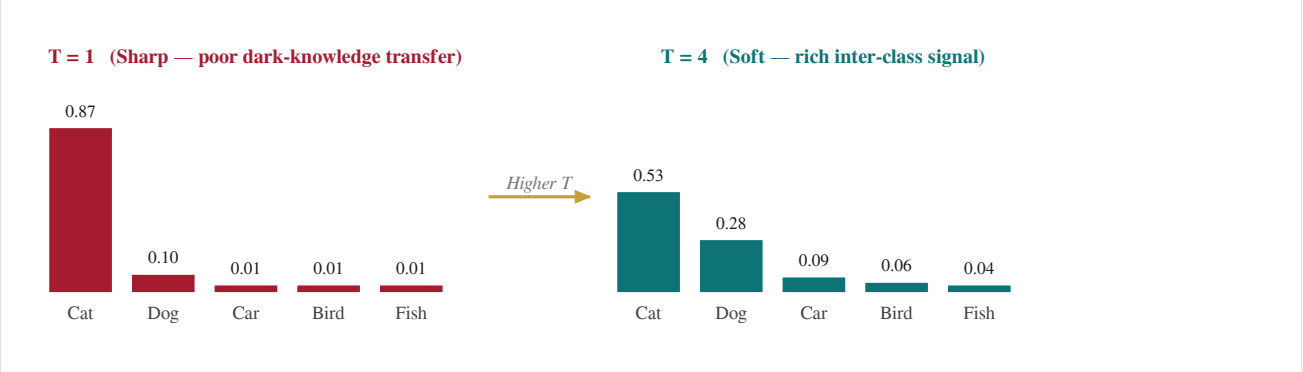

Figure 2.1 — T=1 gives a sharp distribution; T=4 softens it, revealing inter-class similarity as the dark knowledge that enriches the student's training signal.

```
def soft_probabilities(logits: torch.Tensor, T: float) -> torch.Tensor:
    """T=1 → standard softmax;  T>1 → softer, richer gradient signal."""
    return F.softmax(logits / T, dim=-1)
```

2.2 The Combined Distillation Loss

```
class DistillationLoss(nn.Module):
    def __init__(self, alpha=0.5, temperature=4.0):
        super().__init__()
        self.alpha, self.T = alpha, temperature
        self.kl = nn.KLDivLoss(reduction='batchmean')
        self.ce = nn.CrossEntropyLoss()
    def forward(self, s_logits, t_logits, labels):
        kd = self.kl(F.log_softmax(s_logits/self.T,-1),
                     F.softmax(t_logits /self.T,-1)) * self.T**2
        return self.alpha*kd + (1-self.alpha)*self.ce(s_logits, labels)
```

2.3 Why T^2 Scaling?

Temperature-softened gradients are T^2 times smaller than their hard-label counterparts. The T^2 factor in the KL term restores gradient magnitude so both loss components remain on a comparable scale at any T.

© 2025 Ashutosh Sinha. All rights reserved.

3. Taxonomy of Distillation Approaches

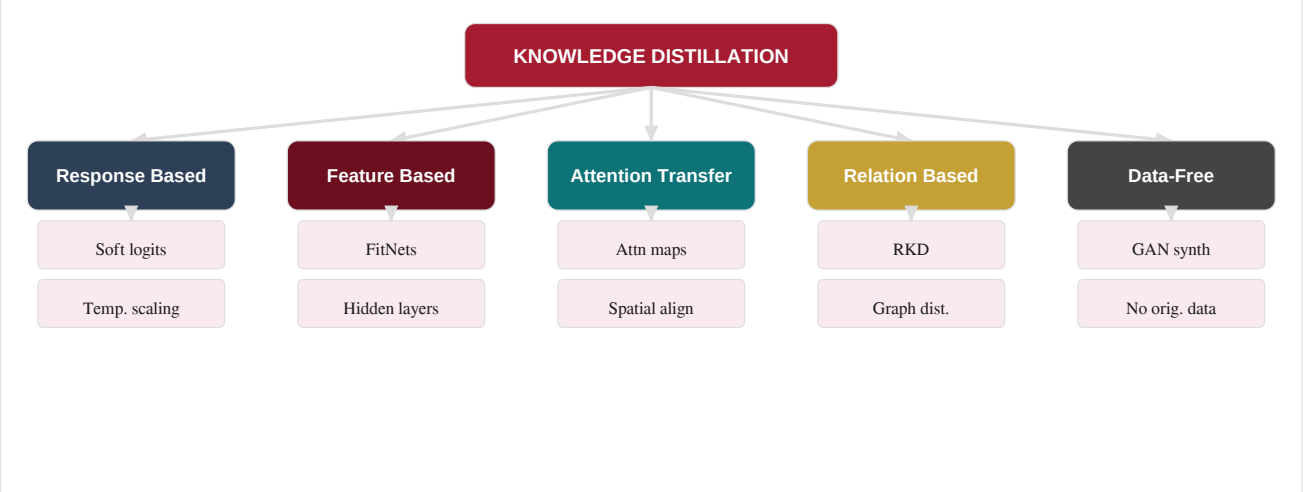

Figure 3.1 — Five distillation families, their signal sources, and representative sub-techniques.

3.1 Response-Based

Mimics teacher final-layer logits. Simplest; effective when teacher and student share similar depth.

3.2 Feature-Based (FitNets)

Intermediate layer activation guidance. Valuable when the student is much shallower.

3.3 Attention Transfer

Distils spatial attention maps from teacher layers. Improves localisation jointly with classification.

3.4 Relation-Based & Data-Free

Relation-based methods distil inter-sample structural relationships — architecture-agnostic. Data-free distillation uses a GAN to synthesise pseudo-samples when no original data is available.

Approach	Signal	Best For	Cost
Response-Based	Output logits	Classification, NLP	Low
Feature-Based	Hidden layers	Shallow students	Medium
Attention Transfer	Spatial attn	Detection, segmentation	Low–Med
Relation-Based	Inter-sample graphs	Arch-agnostic	Medium
Data-Free	GAN data	Privacy domains	High
Online/Mutual	Peer predictions	Collaborative training	Medium

© 2025 Ashutosh Sinha. All rights reserved.

© 2025 Ashutosh Sinha. All rights reserved.

3b. Literature Survey

This section surveys the five research threads that KD-SPAR draws upon and positions against: knowledge distillation, prompt optimisation, self-refinement and meta-cognition, federated learning, and metaprompting.

Knowledge Distillation

Model compression via knowledge distillation was pioneered by Bucilua et al. (2006) [2] and formalised by Hinton, Vinyals & Dean (2015) [1] with the temperature-scaled softmax that remains the standard KD formulation today. Romero et al. (2015) [3] extended KD to intermediate feature layers (FitNets), showing that hidden representations carry information lost in logit-only transfer. Zagoruyko & Komodakis (2017) [5] introduced attention transfer, distilling spatial attention maps for improved localisation. Park et al. (2019) [4] proposed relational KD (RKD), operating on inter-sample structural relationships rather than individual outputs.

For NLP, Sanh et al. (2019) [6] demonstrated that DistilBERT retains 97% of BERT's performance at 40% fewer parameters. Jiao et al. (2020) [7] extended this with TinyBERT's attention-head distillation for 7.5\u00d7 speedup. Beyer et al. (2022) [8] showed that patient, consistent distillation significantly outperforms one-shot approaches. More recently, Microsoft's Phi-3 [9] demonstrated GPT-4-class reasoning in a 3.8B model distilled from synthetic data. Gou et al. (2021) [10] provide a comprehensive survey of the field.

Prompt Optimisation

When model weights are inaccessible (API-only deployment), prompt optimisation is the primary lever for improving model behaviour. OPRO (Yang et al., 2023) [16] treats an LLM as an optimiser: the model proposes prompt candidates scored by task accuracy. APE (Zhou et al., 2023) [18] generates and scores candidate prompts automatically. EvoPrompt (Guo et al., 2023) [17] applies evolutionary search with LLM-driven mutation and crossover. DSPy (Khattab et al., 2024) [14] compiles declarative LLM pipelines into optimised prompts using a task metric as the compilation signal.

All of these methods treat the language model as a black box scored by task accuracy. KD-SPAR differs in three fundamental ways: (1) the optimisation target is the teacher's output distribution (KD divergence), not task accuracy; (2) the model that proposes the instruction is the same model that will execute it (self-authorship); and (3) the student first diagnoses its own failure modes before proposing a fix, rather than generating generic candidates.

Self-Refinement and Meta-Cognition

Self-Refine (Madaan et al., 2023) [20] lets a model iteratively improve its outputs given self-generated feedback. Constitutional AI (Bai et al., 2022) [19] uses self-critique to improve alignment by having the model evaluate its own outputs against a set of principles. Both methods improve outputs given a fixed system prompt. KD-SPAR improves the instructions themselves \u2014 it operates one level of abstraction higher and is therefore more durable: a better prompt benefits all future queries, not just the one being refined.

Federated Learning

© 2025 Ashutosh Sinha. All rights reserved.

FedAvg (McMahan et al., 2017) [22] enables collaborative model training by sharing gradient updates. However, gradient sharing leaks training data through inversion attacks. The Federated KD-SPAR variant communicates only natural-language instruction strings \u2014 a strictly weaker information channel than gradient vectors \u2014 providing a stronger practical privacy guarantee for multi-site deployments.

Metaprompting

Metaprompting (Suzgun & Kalai, 2024) orchestrates a single LLM through a conductor \u2192 specialist architecture: a conductor prompt decomposes a task, delegates to expert personas (each with a domain-specific system prompt), and synthesises their outputs. The key insight is that one model wearing different hats can outperform the same model with a single monolithic prompt. MetaKDSPAR (Section 13b) integrates this insight into the KD-SPAR diagnostic loop: four specialist perspectives independently analyse student failures, a conductor synthesises the diagnoses, and each specialist proposes fixes from its domain. This combination of metaprompting + KD as the objective signal + student self-authorship has not been explored in the literature.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

PART II

Implementation & Techniques

Code · Pipelines · Advanced Methods

4. End-to-End Implementation

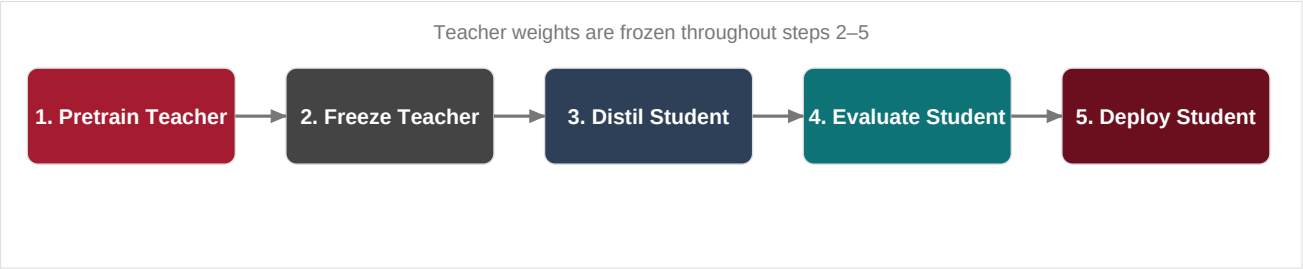

Figure 4.1 — Five-stage distillation pipeline: pretrain, freeze, distil, evaluate, deploy.

4.1 ResNet-50 → MobileNetV2 on CIFAR-10

```
teacher = models.resnet50(pretrained=True); teacher.fc = nn.Linear(2048,10); teacher.eval()
student = models.mobilenet_v2(pretrained=False); student.classifier[1] = nn.Linear(1280,10)
criterion = DistillationLoss(alpha=0.6, temperature=4.0)
optimizer = torch.optim.Adam(student.parameters(), lr=1e-3)
for epoch in range(30):
    for images,labels in loader:
        with torch.no_grad(): t_logits = teacher(images.to(DEVICE))
        loss = criterion(student(images.to(DEVICE)), t_logits, labels.to(DEVICE))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
```

4.2 BERT → DistilBERT

```
class BertDistillLoss(nn.Module):
    def forward(self, s_logits, t_logits, s_hidden, t_hidden, labels):
        kd = kl_div(log_softmax(s_logits/T,-1), softmax(t_logits/T,-1)) * T**2
        hs = mse(s_hidden, t_hidden.detach())
        ce = cross_entropy(s_logits, labels)
        return alpha*kd + beta*hs + (1-alpha)*ce
```

© 2025 Ashutosh Sinha. All rights reserved.

5. Advanced Techniques

5.1 Progressive / Multi-Stage

Each stage reduces capacity by a controlled factor. The student from stage N becomes the teacher for stage N+1. Avoids the capacity-gap failure mode in one-shot large-to-tiny jumps.

5.2 Online / Mutual Learning

```
# Two students teach each other simultaneously
for images, labels in loader:
    l1, l2 = s1(images), s2(images)
    loss1 = 0.5*ce(l1,labels) + 0.5*kl(log_softmax(l1,-1), softmax(l2.detach(),-1))
    loss2 = 0.5*ce(l2,labels) + 0.5*kl(log_softmax(l2,-1), softmax(l1.detach(),-1))
    for opt,loss in [(opt1,loss1),(opt2,loss2)]:
        opt.zero_grad(); loss.backward(retain_graph=True); opt.step()
```

5.3 Self-Distillation

The deepest exit acts as the teacher for shallower exits in the same network. No separate teacher model required; consistently improves generalisation.

© 2025 Ashutosh Sinha. All rights reserved.

PART III

Applications & Deployment

Real-World Cases · Benchmarks

6. Real-World Applications

6.1 On-Device NLP — DistilBERT & TinyBERT

DistilBERT retains 97% of BERT's GLUE performance at 40% smaller and 60% faster. TinyBERT adds attention-head distillation for a 7.5× speedup.

6.2 Real-Time Object Detection — YOLOv8

YOLOv8-Nano is distilled from YOLOv8-X. Soft bounding-box targets sharpen localisation accuracy at a 30× size reduction.

6.3 LLM Compression — Phi-3 Mini

Microsoft distilled a 3.8B-parameter model that outperforms Llama-2-7B on reasoning, using GPT-4-generated synthetic data as both training signal and soft targets.

6.4 Medical Imaging — Privacy-Preserving Distillation

A generator produces plausible X-ray patches from aggregate statistics. A small model is distilled without transmitting raw patient data — satisfying HIPAA and GDPR simultaneously.

6.5 Edge Speech — Whisper Compression

Whisper-large distilled into Whisper-tiny using token-probability sequences as targets. 150 MB, real-time on Raspberry Pi 4, WER increase under 3 pp.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

7. Benchmarks & Published Results

Model Pair	Task	Teacher	No KD	With KD	Compression
ResNet-50 → MobileNetV2	CIFAR-10 Acc.	93.4%	87.2%	91.8%	4× smaller
BERT-base → DistilBERT	GLUE avg.	84.0	77.0	81.8	2× faster
BERT-large → TinyBERT	GLUE avg.	87.0	—	82.5	7.5× faster
YOLOv8-X → YOLOv8-N	COCO mAP	53.9%	37.3%	41.2%	30× smaller
GPT-4 → Phi-3-Mini	MT-Bench	8.7	—	8.1	100× smaller
Whisper-L → Whisper-tiny	WER LibriSpeech	2.7%	10.3%	5.6%	32× smaller

Interpretation

KD consistently recovers 50–80% of the teacher accuracy gap. The 'No KD' → 'With KD' delta isolates the pure value of distillation independent of architectural improvements.

© 2025 Ashutosh Sinha. All rights reserved.

PART IV

RAG & Model Migration

Behavioural Transfer · Prompt KD Foundations

8. KD When Your RAG Model Changes

In a production custom RAG pipeline the generative model at the end of the retrieval chain is rarely permanent. Model transitions cause silent regressions: citation formats drift, reasoning depth drops, confidence language changes, and downstream parsers break.

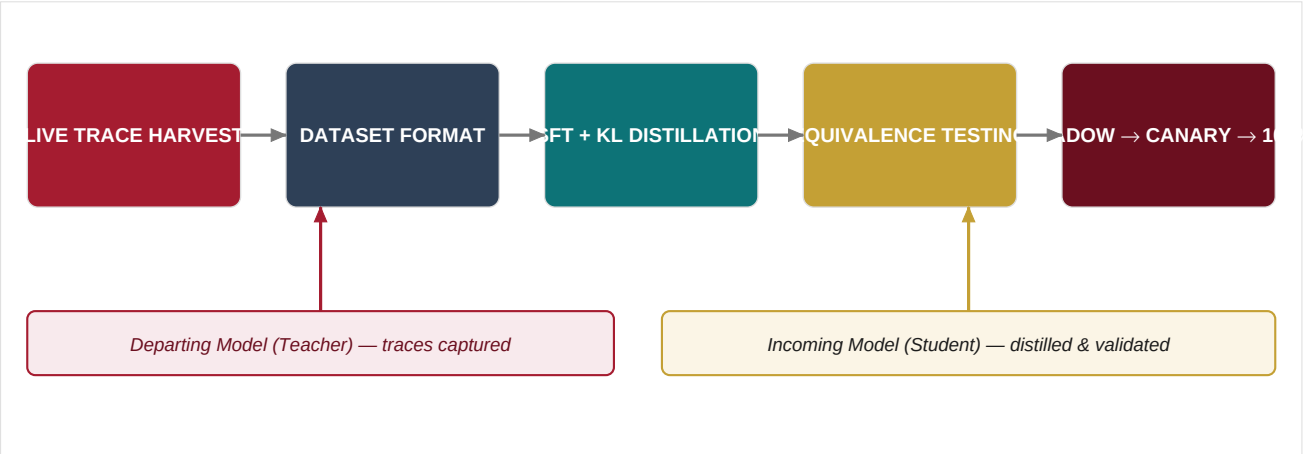

Figure 8.1 — Five-phase RAG model migration using KD as the behavioural transfer mechanism from departing teacher to incoming student.

Architecture Context

In a multi-model RAG stack (LangChain / LlamaIndex, enterprise multi-cloud, query-router dispatching to GPT-4o, Claude, and open-source models), each route has its own implicit behavioural contract. KD transfers that contract to the incoming model before it ever handles live traffic.

8.1 Why Transitions Silently Break Pipelines

- Answer format regression: citation patterns change, breaking downstream parsers.
- Retrieval-generation alignment shift: new model weights retrieved passages differently.
- Calibration drift: uncertainty language changes, misfiring escalation thresholds.
- Chain-of-thought fragmentation: multi-hop reasoning paths collapse under the new model.

8.2 Harvesting Teacher Traces & Routing-Aware Distillation

```
# Partition traces by query route; each route-student is distilled separately
```

© 2025 Ashutosh Sinha. All rights reserved.

```
for route, route_traces in partition_by_route(all_traces, router).items():
    run_sft(student_model, Dataset.from_list(route_traces),
            output=f'./distilled/{route}')
```

8.3 Behavioural Equivalence Gates

Check	Measures	Pass Threshold
Citation Fidelity	% responses citing [Doc-N] when teacher did	> 90% of teacher
Semantic Similarity	BERTScore F1 vs. teacher	≥ 0.85
Calibration	Hedging-phrase frequency ratio	0.80 – 1.20
Format Preservation	Structured response parse rate	100%
Hallucination Proxy	1 – citation-when-cited rate	≤ 0.12

© 2025 Ashutosh Sinha. All rights reserved.

9. KD for Prompt Tuning — When Fine-Tuning Is Not Possible

When foundation model weights are inaccessible (API-only, proprietary, compliance), prompt tuning guided by KD is the practical alternative. The teacher's output distribution is the optimisation target; the prompt is adjusted rather than weights.

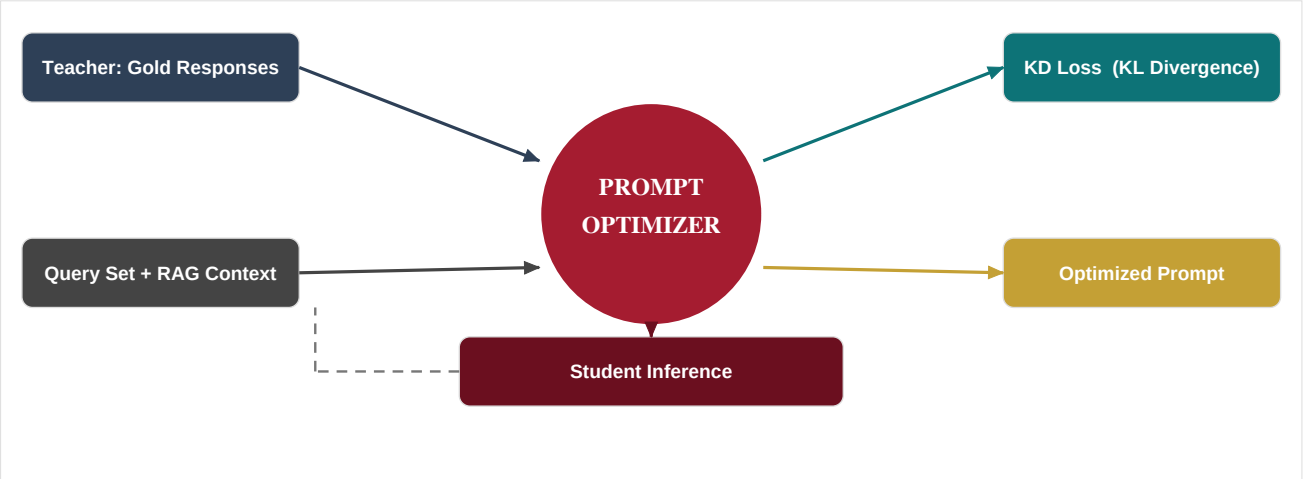

Figure 9.1 — KD-guided prompt optimisation: the teacher generates gold responses; the KD loss drives iterative refinement of the student's prompt.

9.1 Soft Prompt Distillation

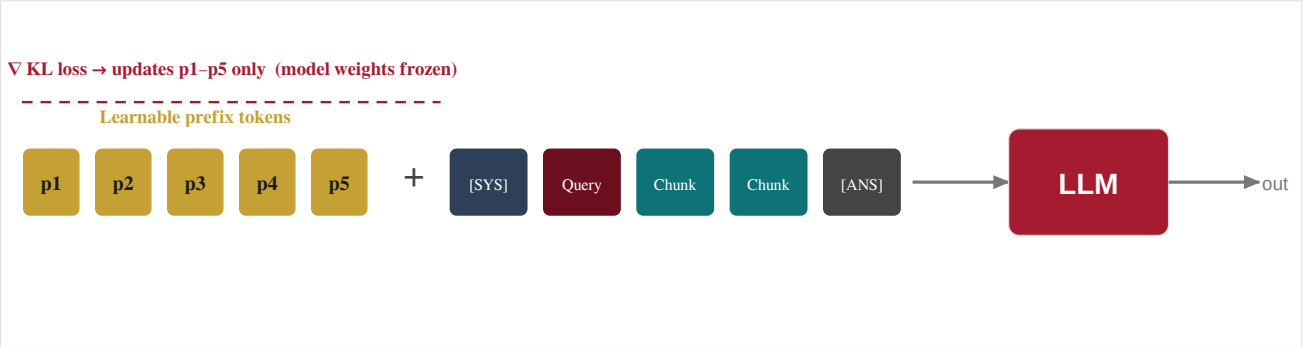

Figure 9.2 — Learnable prefix tokens p1–p5 are optimised to minimise KL divergence to teacher; all LLM weights remain frozen (PEFT PrefixTuning).

```
from peft import get_peft_model, PrefixTuningConfig, TaskType
peft_cfg = PrefixTuningConfig(task_type=TaskType.CAUSAL_LM, num_virtual_tokens=20)
student  = get_peft_model(AutoModelForCausalLM.from_pretrained('mistral-7b'), peft_cfg)
# trainable params: ~5M out of 7.2B  (0.07%)
```

9.2 Evolutionary Prompt Search & DSPy Compilation

© 2025 Ashutosh Sinha. All rights reserved.

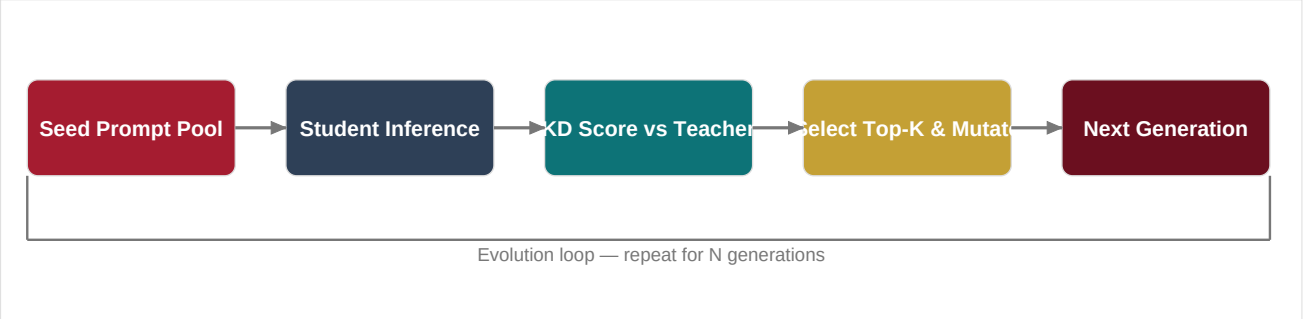

Figure 9.3 — Evolutionary APO: KD score is the fitness function; mutation and crossover evolve prompts towards teacher alignment.

© 2025 Ashutosh Sinha. All rights reserved.

PART V

KD-SPAR Variants

Base · Multi-Teacher · Adversarial · Federated

10. KD-SPAR: Student-Driven Prompt Auto-Rewriting

Sections 8 and 9 treat prompt optimisation as an external operation. KD-SPAR proposes a fundamentally different paradigm: the student model itself diagnoses where its outputs diverge from the teacher and writes targeted amendments to its own system prompt. This self-calibrating loop requires no weight updates and no external optimiser beyond the student's own generative capability.

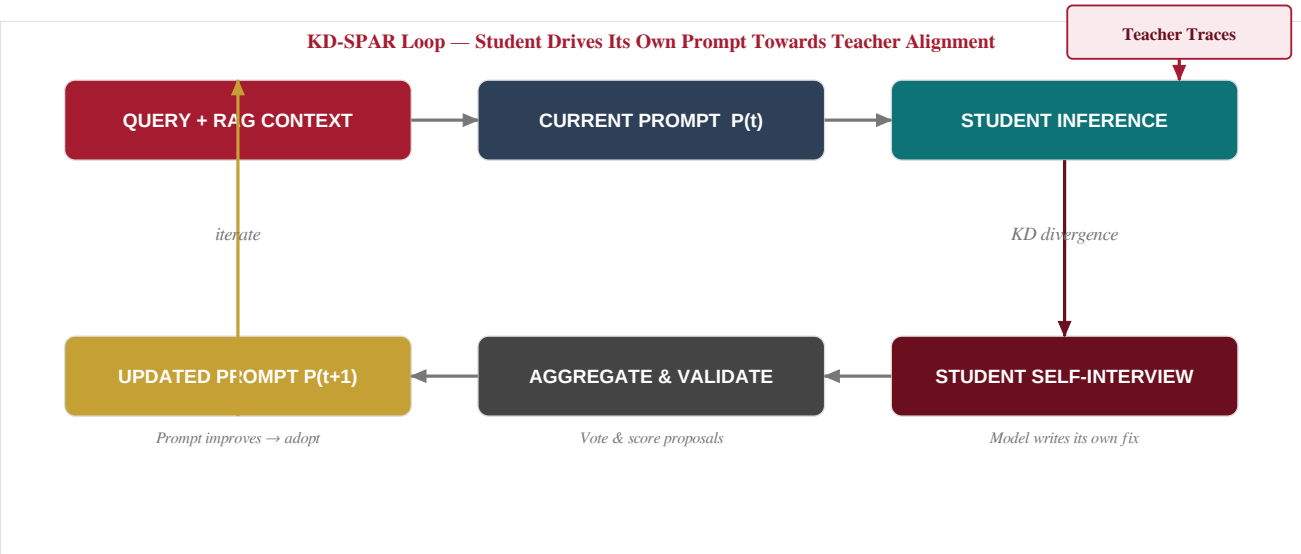

Figure 10.1 — KD-SPAR feedback loop: the student proposes prompt rewrites based on its diagnosed failure modes and iterates until teacher-alignment converges.

10.1 The Core Insight: Student Self-Knowledge

The student has privileged access to its own internal representations. It knows what types of instructions elicit what kinds of responses. Standard prompt optimisers (evolutionary search, DSPy, soft prompts) do not exploit this self-knowledge. KD-SPAR does: the student is asked, in natural language, to reason about its own failure and propose a concrete instruction that would close the gap.

Why KD-SPAR Is Different

Standard APO treats the model as a black box and uses KD loss as a fitness signal. KD-SPAR treats the model as an active agent with self-knowledge. The student is not just evaluated by the KD signal — it reads the signal and writes its own fix. This makes KD-SPAR self-calibrating in a way that no external optimiser can be.

© 2025 Ashutosh Sinha. All rights reserved.

10.2 The Four-Phase Algorithm

- Phase 1 — Diagnostic Pass: run student on training queries, score against teacher traces using BERTScore (or Jaccard fallback), classify top-k failures by mode (missing_citation, over_hedged, under_hedged, incomplete, format_drift).
- Phase 2 — Self-Interview: construct a meta-prompt showing the student its own failure and the target pattern; ask it to propose one actionable instruction per failure mode. Generate N proposals per failure.
- Phase 3 — Aggregation: cluster proposals semantically (AgglomerativeClustering over MiniLM embeddings); score each cluster representative on a mini eval set; select top-K instructions by KD score improvement.
- Phase 4 — Validate & Commit: compose the candidate prompt; run the full equivalence suite on held-out queries; accept only if aggregate KD score improves by threshold δ. Otherwise revert.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

11. Multi-Teacher KD-SPAR

When migrating from a committee of specialist models to a single general-purpose student — for example, a citation-expert Claude, a reasoning-depth GPT-4o, and a calibration-specialist Gemini — the student must satisfy all teachers simultaneously. Multi-Teacher KD-SPAR extends the base algorithm to handle N teachers with independent KD scores.

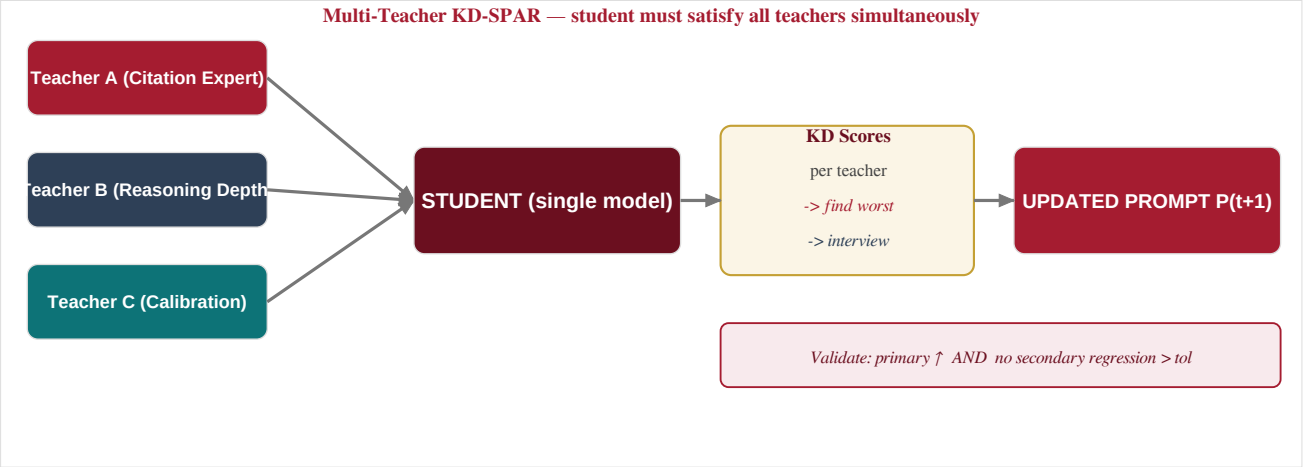

Figure 11.1 — Multi-Teacher KD-SPAR: three specialist teachers independently score the student; the worst-aligned teacher drives the self-interview for that iteration; validation requires primary improvement AND no secondary regression.

11.1 The Worst-Teacher Principle

At each iteration, compute the student's KD score against every teacher. The teacher with the lowest weighted alignment score — the worst teacher for this iteration — provides the target for the self-interview prompt. This ensures that all teachers receive attention over successive iterations rather than the student becoming aligned with only the primary teacher.

11.2 Non-Regression Validation

A candidate prompt is accepted only when two conditions hold simultaneously: (a) the primary teacher's KD score improves by at least the threshold δ, and (b) no secondary teacher's score regresses by more than a tolerance τ (typically 2%). This prevents the student from over-fitting one teacher's style at the expense of another.

```
# Multi-Teacher validation gate
accepted = (primary_delta >= threshold) and all(
    new_scores[t.name] >= old_scores[t.name] - regression_tol
    for t in self.teachers
)
```

11.3 Configuration Example

```
from sara.rag.kd_spar_multi_teacher import MultiTeacherKDSPAR, TeacherSpec
```

© 2025 Ashutosh Sinha. All rights reserved.

```
specs = [
    TeacherSpec('citation',  'claude-3-5-sonnet-20241022', weight=2.0, is_primary=True),
    TeacherSpec('reasoning', 'gpt-4o',                     weight=1.5),
    TeacherSpec('calibrate', 'gemini-1.5-pro',             weight=1.0),
]
spar = MultiTeacherKDSPAR(
    student_model  = 'claude-sonnet-4-5-20250929',
    teachers       = specs,
    vector_store   = store,
    regression_tol = 0.02,
)
# Harvest responses from all teachers
teacher_response_sets = spar.harvest_teacher_responses(query_log)
# Run the multi-teacher loop
final_prompt, history = spar.run(
    train_queries, val_queries, teacher_response_sets,
    iterations=10, threshold=0.003,
)
```

© 2025 Ashutosh Sinha. All rights reserved.

12. Adversarial KD-SPAR

Standard KD-SPAR optimises on the query distribution seen in production logs. However, the long tail of hard, edge-case, and out-of-distribution queries may remain poorly handled. Adversarial KD-SPAR focuses the optimisation loop exclusively on hard examples — queries where the teacher-student KD gap is largest, or queries specifically designed by the teacher model to expose the student's weakest failure modes.

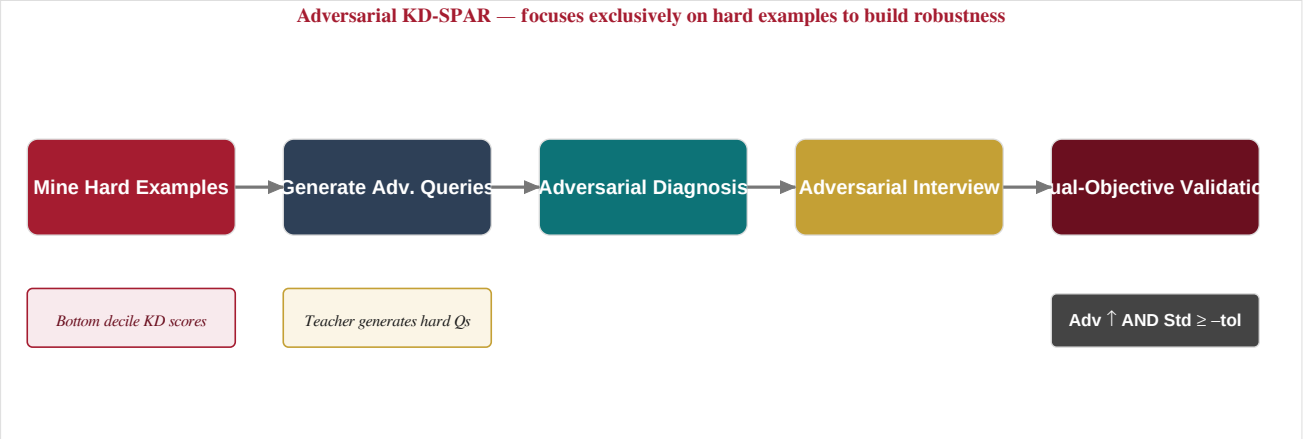

Figure 12.1 — Adversarial KD-SPAR: hard examples are either gap-mined from production logs (bottom percentile KD scores) or generated by the teacher. Dual-objective validation requires improvement on hard queries AND no regression on standard queries.

12.1 Two Sources of Hard Examples

Hard examples arrive from two complementary sources. Gap-mined examples are production queries where the student already exhibits the largest KD divergence — the bottom decile of KD scores across all logged queries. These are real failures with known context. Adversarially generated examples are produced by the teacher model using a structured prompt that requests questions requiring multi-hop reasoning, contradiction handling, edge-case awareness, and boundary testing. These anticipate failures before they appear in production.

12.2 Dual-Objective Validation

A naive adversarial optimiser risks over-fitting to the hard examples and regressing on common queries. Adversarial KD-SPAR prevents this with a dual threshold: the candidate prompt must improve on adversarial queries by at least δ_{adv} AND not degrade standard queries by more than the regression tolerance τ_{std}.

```
from sara.rag.kd_spar_adversarial import AdversarialKDSPAR

spar = AdversarialKDSPAR(
    teacher_model         = 'claude-3-5-sonnet-20241022',
    student_model         = 'claude-sonnet-4-5-20250929',
    vector_store          = store,
    adversarial_topics    = ['knowledge distillation', 'RAG retrieval'],
    n_generated_per_topic = 10,
    hardness_percentile   = 0.25,   # bottom 25% of KD scores
    dual_threshold        = 0.005,
```

© 2025 Ashutosh Sinha. All rights reserved.

```
    standard_regression   = 0.02,
)
hard_queries = spar.build_hard_query_set(production_queries, teacher_responses)
final_prompt, history = spar.run_adversarial(
    adversarial_queries = hard_queries,
    standard_queries    = production_queries,
    teacher_responses   = teacher_responses,
    iterations          = 8,
)
```

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

13. Federated KD-SPAR

Many real-world deployments involve multiple organisationally or geographically distributed student instances — hospital branches, regional offices, department deployments — each with private, locally held RAG data that cannot be shared. Federated KD-SPAR enables all these sites to jointly optimise a single global student prompt without any raw data leaving any site.

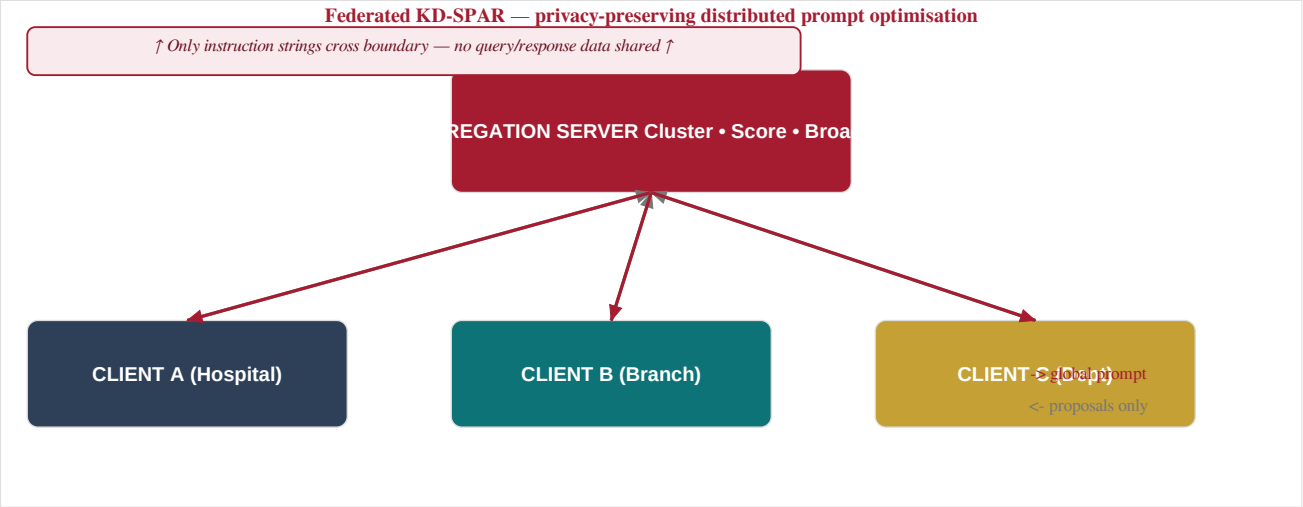

Figure 13.1 — Federated KD-SPAR: clients run diagnosis and self-interview locally; only proposed instruction strings (never query/response data) are sent to the aggregation server; the server validates and broadcasts an updated global prompt.

13.1 Privacy Guarantee

The only information that crosses the client→server boundary is a list of natural-language instruction strings proposed by each client's student model (e.g. 'Always cite retrieved passages using [Doc-N] notation'). No query text, no retrieved document content, and no model response data leave each client site. This is a stronger privacy guarantee than federated gradient sharing, which can leak training data through gradient inversion attacks.

13.2 Federated Round Protocol

- Broadcast: Server sends current global prompt P_t to all clients.
- Local Diagnosis: Each client runs KD-SPAR diagnosis on its private RAG traces, generates proposed instructions, applies a local pre-filter (top-K by local mini-eval), and sends only the instruction strings to the server.
- Aggregation: Server clusters all proposals semantically, scores each cluster representative against its server-side validation queries, selects top-K.
- Validation & Broadcast: Server tests the candidate P_{t+1}, accepts if server KD score improves by δ, broadcasts P_{t+1} (or P_t if rejected).

13.3 Implementation

```
from sara.rag.kd_spar_federated import FederatedSimulation
```

© 2025 Ashutosh Sinha. All rights reserved.

```
# Simulate 3 client sites from a shared trace pool
sim    = FederatedSimulation(
    n_clients   = 3,
    all_traces  = [(q, teacher_resp) for q,teacher_resp in trace_pairs],
    student_model = 'claude-sonnet-4-5-20250929',
    vector_store  = store,
)
server = sim.build_server(threshold=0.003, regression_tol=0.02)
final_prompt, history = server.run(rounds=10)
# Production: deploy real distributed clients
from sara.rag.kd_spar_federated import (
    FederatedKDSPARClient, FederatedKDSPARServer, FederatedClientConfig
)
clients = [
    FederatedKDSPARClient(
        FederatedClientConfig(client_id='hospital_a'),
        local_traces=site_a_traces, vector_store=site_a_store
    ),
    # ... more clients
]
server  = FederatedKDSPARServer(
    clients=clients, server_val_queries=val_q,
    server_val_responses=val_resps, student_model=STUDENT_MODEL,
)
final_prompt, history = server.run(rounds=10)
```

13.4 When to Use Each KD-SPAR Variant

Variant	Trigger	Key Constraint	Privacy
Base KD-SPAR	Single teacher, common queries	None	Single site
Multi-Teacher	Committee of specialist teachers	Non-regression across teachers	Single site
Adversarial	Long-tail robustness needed	Dual-objective validation	Single site
Federated	Multi-site, private local data	No data sharing	Multi-site
MetaKDSPAR	Compound failures, richer diagnosis	Higher inference cost	Single site

© 2025 Ashutosh Sinha. All rights reserved.

13b. MetaKDSPAR: Metaprompting-Enhanced Prompt Distillation

Standard KD-SPAR uses a single monolithic diagnostic pass: one call to `_classify_failure()` categorises the failure into one of five modes, and one self-interview prompt generates proposals. This works well for simple failures but misses compound failures — for example, a response that is simultaneously under-hedged and missing citations. MetaKDSPAR addresses this limitation using a conductor + specialist architecture inspired by metaprompting (Suzgun & Kalai, 2024).

13b.1 Architecture

MetaKDSPAR decomposes the diagnostic and proposal phases into four specialist perspectives, each implemented as a distinct system prompt on the same student model:

- Citation Specialist: focuses exclusively on [Doc-N] notation, citation placement, and unsupported claims.
- Calibration Specialist: analyses hedging language — detects over-hedged and under-hedged responses relative to the teacher.
- Completeness Specialist: compares information coverage, depth, and reasoning chain length against the teacher response.
- Format Specialist: analyses structural alignment — paragraph organisation, tone, list/prose balance, and presentation style.

A conductor prompt synthesises the specialist diagnoses, ranks them by impact, and selects the top-K failures for the proposal phase. Each selected failure is then addressed by its diagnosing specialist, who proposes a targeted instruction from its domain expertise. The conductor reconciles cross-specialist proposals before the standard validate-and-commit gate.

13b.2 Algorithm Differences from Base KD-SPAR

Phase	Base KD-SPAR	MetaKDSPAR
Diagnosis	Single _classify_failure() call	K specialist prompts + conductor synthesis
Self-Interview	One generic interview prompt	Specialist-perspective proposals per domain
Aggregation	Score all proposals on mini-eval	Conductor reconciles cross-specialist proposals
Validation	Same validate-and-commit gate	Same validate-and-commit gate

The validate-and-commit gate is identical to base KD-SPAR — MetaKDSPAR changes only how failures are diagnosed and proposals generated, not the acceptance criterion. This ensures a fair comparison in the ablation: Condition E (MetaKDSPAR) vs. Condition A (base KD-SPAR) isolates the value of multi-perspective diagnosis.

13b.3 Testable Hypothesis

MetaKDSPAR Hypothesis

© 2025 Ashutosh Sinha. All rights reserved.

Multi-perspective self-diagnosis catches compound failures that flat single-pass diagnosis misses, producing higher-quality prompt amendments. The E\u2212A gap (MetaKDSPAR minus base KD-SPAR) quantifies this advantage. A positive E\u2212A gap, controlling for the same student model, KD metric, and validate-and-commit gate, would demonstrate that the conductor + specialist architecture adds diagnostic value beyond what a monolithic classifier provides.

13b.4 Implementation

```
from sara.rag.kd_spar_meta import MetaKDSPAR

meta = MetaKDSPAR(
    student_model='llama3.2:3b',
    vector_store=store,
    # Uses 4 built-in specialists by default
)

final_prompt, history = meta.run(
    train_queries=train_q,
    val_queries=val_q,
    teacher_responses=teacher_responses,
    iterations=3,
    top_k_diag=3,     # conductor selects top-3 diagnoses
    top_k_instr=3,    # top-3 instructions per iteration
)
```

13b.5 Trade-offs

Cost: Each MetaKDSPAR iteration requires K specialist diagnostic calls (K=4 by default) plus one conductor call per query, compared to one diagnostic call in base KD-SPAR. This is a 5\u00d7 increase in Phase 1 inference cost. The proposal phase also increases because each specialist generates proposals from its perspective. With local Ollama models the cost is wall-clock time only; with API models the per-call cost multiplies accordingly.

When to use MetaKDSPAR: When the student's failures are consistently compound (multiple failure modes per query), when base KD-SPAR iterations plateau without improving, or when the student model is large enough (7B+) to maintain distinct specialist perspectives effectively. For small students (3B), the overhead may not be justified.

© 2025 Ashutosh Sinha. All rights reserved.

PART VI

Practitioner Reference

Hyperparams · Evaluation · Best Practices · Frameworks

14. Hyperparameter Selection Guide

14.1 Temperature (T) Sweep Utility

```
def temperature_sweep(teacher, student, loader, temps=[2,3,4,5,6,8], device=DEVICE):
    results = {}
    for T in temps:
        acc = train_and_eval(student, teacher, loader, DistillationLoss(0.5,T), device, 5)
        results[T] = acc; print(f'T={T}  val={acc:.4f}')
    return results
# Vision: T in {2-6}   |   NLP: T in {4,8,16}
```

14.2 Selection Table

Scenario	T	Alpha (α)	Notes
Large dataset, close capacity	2–4	0.4–0.6	Standard
Small dataset, wide gap	4–8	0.6–0.8	Lean on teacher
NLP / large vocabulary	8–16	0.5–0.7	Many near-zero logits
Data-free (synthetic)	4–6	0.8–1.0	Hard labels unreliable
Self-distillation	1–3	0.3–0.5	Same capacity

© 2025 Ashutosh Sinha. All rights reserved.

15. Evaluation & Quality Metrics

15.1 Three-Number Reporting Rule

Always report: (a) teacher on full test set, (b) student hard-labels only, (c) student with KD. The delta (b→c) isolates the pure KD contribution.

15.2 Efficiency Profiler

```
def profile_model(model, dummy, n=200):
    model.eval()
    with torch.no_grad():
        for _ in range(20): model(dummy)
    t0 = time.perf_counter()
    with torch.no_grad():
        for _ in range(n): model(dummy)
    lat = (time.perf_counter()-t0)/n*1000
    mem = sum(p.numel()*p.element_size() for p in model.parameters())/1e6
    print(f'Latency: {lat:.2f}ms  Params: {mem:.1f}MB')
```

© 2025 Ashutosh Sinha. All rights reserved.

16. Best Practices & Common Pitfalls

16.1 Best Practices

- Freeze the teacher and wrap its forward pass in `torch.no_grad()`.
- Use cosine-annealing or warm-up LR — distillation loss is noisy in early epochs.
- Validate teacher quality first: a weak teacher produces misleading soft targets.
- Combine KD with MixUp / CutMix / RandAugment for additional regularisation.
- Add a projection head when teacher and student feature dims differ.
- Log KD loss and CE loss separately to diagnose balance issues early.

16.2 Common Pitfalls

- T too high: near-uniform distribution eliminates discriminative signal.
- Alpha too high on small data: student overfits teacher's idiosyncratic errors.
- Missing T^2 scaling: KL term becomes negligible at $T > 1$ without it.
- Capacity mismatch: a student too small cannot absorb the information.
- Stale batch-norm stats: re-calibrate BN running means after distillation.

© 2025 Ashutosh Sinha. All rights reserved.

17. Framework Integrations

17.1 PyTorch Lightning

```
class DistillModule(pl.LightningModule):
    def training_step(self, batch, _):
        x, y = batch
        with torch.no_grad(): tl = self.teacher(x)
        loss = self.criterion(self.student(x), tl, y)
        self.log('loss', loss, prog_bar=True)
        return loss
pl.Trainer(max_epochs=30, accelerator='auto').fit(
    DistillModule(teacher, student), train_loader, val_loader)
```

17.2 Hugging Face Trainer

```
class DistillTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kw):
        labels = inputs.pop('labels')
        s = model(**inputs)
        with torch.no_grad(): t = self.teacher(**inputs)
        kd = kl_div(log_softmax(s.logits/self.T,-1),
                    softmax(t.logits /self.T,-1),'batchmean') * self.T**2
        loss = self.alpha*kd + (1-self.alpha)*cross_entropy(s.logits,labels)
        return (loss, s) if return_outputs else loss
```

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

PART VII

Frontier & Future Directions

LLM Distillation · Multimodal · Extensions

18. Future Directions

18.1 LLM-to-LLM Distillation at Scale

Distillation from GPT-4/Claude-class models to 7B–13B students is the primary route to capable, deployable AI. Open challenges include distribution shift, reward-model alignment during distillation, and token-level calibration across tokeniser mismatches.

18.2 Multimodal & Quantisation-Aware Distillation

Distilling vision-language models requires simultaneous alignment of visual encoder representations and language decoder logits. Combining with 4-bit QAT produces models that are simultaneously compressed, quantised, and behaviourally aligned.

18.3 KD-SPAR Extensions

- Adversarial SPAR with active learning: the student proposes queries it is most uncertain about, the teacher labels them, and these become adversarial examples for the next iteration.
- Federated SPAR with differential privacy: add calibrated Laplace noise to proposed instruction embeddings before sending to the server to provide (ε, δ)-differential privacy.
- Multi-teacher SPAR with dynamic weighting: dynamically re-weight teachers each iteration based on which teacher's distribution is currently closest to the student's — allocating interview effort where the gap is largest.
- MetaKDSPAR with learned specialist selection: instead of running all K specialists on every query, train a lightweight router that predicts which specialist perspectives are most likely to be informative for a given failure pattern — reducing the inference overhead from K× to ~2× while preserving diagnostic quality.
- MetaKDSPAR with hierarchical conductors: for complex multi-hop RAG tasks, a two-level conductor hierarchy could decompose the task into sub-queries, diagnose each sub-query with specialists, and then synthesise at the task level.
- Formal evaluation benchmark: a controlled benchmark comparing KD-SPAR against APO, DSPy, and soft prompts on standardised RAG workloads with known teacher-student pairs.

© 2025 Ashutosh Sinha. All rights reserved.

18b. Related Work

A comprehensive literature survey covering all five research threads (KD, prompt optimisation, self-refinement, federated learning, metaprompting) is provided in Section 3b. This section focuses on the specific points of differentiation between KD-SPAR and prior art.

Prompt Optimisation

OPRO (Yang et al., 2023) treats an LLM as an optimiser that proposes prompt candidates, evaluated by a task-accuracy score. APE (Zhou et al., 2023) generates and scores candidate prompts automatically. EvoPrompt (Guo et al., 2023) applies evolutionary search with LLM-driven mutation. DSPy (Khattab et al., 2024) compiles declarative pipelines into optimised prompts using a task metric as signal. All these methods treat the language model as a black box scored by task accuracy. Sara's KD-SPAR differs in three ways: (1) the optimisation target is the teacher's output distribution (KD divergence), not task accuracy; (2) the model that proposes the instruction is the same model that will execute it (self-authorship); and (3) the student first diagnoses its own failure modes before proposing a fix.

Self-Refinement and Constitutional AI

Self-Refine (Madaan et al., 2023) lets a model iteratively improve its outputs given self-generated feedback. Constitutional AI (Bai et al., 2022) uses self-critique to improve alignment. Both methods improve outputs given a fixed system prompt. KD-SPAR improves the instructions themselves — it operates one level of abstraction higher and is therefore more durable: a better prompt benefits all future queries, not just the one being refined.

Knowledge Distillation for NLP

DistilBERT (Sanh et al., 2019) and TinyBERT (Jiao et al., 2020) demonstrate KD for encoder models. Beyer et al. (2022) show that patient, consistent distillation significantly outperforms one-shot approaches. None of these methods apply KD to the prompt optimisation problem in API-only RAG settings where model weights are inaccessible.

Federated Learning

FedAvg (McMahan et al., 2017) enables collaborative model training by sharing gradients. Gradient-sharing leaks training data through inversion attacks. The Federated KD-SPAR variant communicates only instruction strings — a strictly weaker channel than gradient vectors — providing a stronger practical privacy guarantee for multi-site deployments.

© 2025 Ashutosh Sinha. All rights reserved.

18c. Algorithm: KD-SPAR Formal Specification

The following pseudocode gives a formal specification of the base KD-SPAR algorithm. The three variants (Multi-Teacher, Adversarial, Federated) modify only the shaded phases indicated.

Algorithm 1 — KD-SPAR (Sara Base Variant)

```
Input:  train_queries Q_train, val_queries Q_val
        teacher_responses T = {q: teacher(q) for q ∈ Q_train}
        base_system_prompt P₀, iterations N, threshold δ
Output: optimised prompt P*

P ← P₀
for t = 1 … N do:
  ── Phase 1: Diagnostic Pass ──────────────────────────
  for q ∈ Q_train do:
    s_q ← student.query(q, prompt=P)
    score_q ← KD_score(s_q, T[q])
    mode_q  ← classify_failure(s_q, T[q])
  F ← top-k failures sorted by score_q ascending
  ── Phase 2: Self-Interview ───────────────────────────
  Proposals ← []
  for (q, s_q, T[q], mode_q) ∈ F do:
    prompt_meta ← SELF_INTERVIEW_TEMPLATE(q, s_q, T[q], mode_q)
    proposals ← student.generate(prompt_meta, n=n_proposals)
    Proposals ← Proposals ∪ proposals
  ── Phase 3: Aggregation ──────────────────────────────
  Clusters ← AgglomerativeCluster(embed(Proposals))
  Top ← top-K cluster representatives by mini-eval KD score
  ── Phase 4: Validate & Commit ────────────────────────
  P_candidate ← P + Top
  Δ ← mean_KD(P_candidate, Q_val, T) − mean_KD(P, Q_val, T)
  if Δ ≥ δ then:  P ← P_candidate   ▷ commit
  else:            revert            ▷ reject
return P
```

Variant modifications: Multi-Teacher replaces Phase 1 with per-teacher diagnosis and adds a non-regression gate in Phase 4. Adversarial replaces Phase 1 queries with gap-mined hard examples. Federated distributes Phases 1–2 to client sites and runs Phases 3–4 on the server, sharing only Proposals (instruction strings, never query data).

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

18d. Limitations

- Metric quality. The primary evaluation metric (Jaccard token overlap, or BERTScore F1 as an upgrade) is an automatic proxy. High Jaccard does not guarantee high semantic quality. The ablation should be supplemented with human evaluation (3 raters, Cohen's $\kappa > 0.6$) before journal submission.
- Self-interview reliability. The self-interview phase assumes the student model has reliable meta-cognitive ability — that it can articulate what instructions would improve its own performance. For very weak student models (< 3B parameters), this assumption may fail and Condition B (external proposal) may outperform Condition A.
- Single-domain evaluation. All experiments use a RAG QA setting. Whether the self-knowledge advantage generalises to code generation, structured extraction, or creative writing tasks is not yet established.
- No weight access required — but also no gradient signal. KD-SPAR operates in a discrete prompt space. A continuous optimiser with weight access (soft prompt distillation, LoRA) can use gradient descent, which is more efficient per update. KD-SPAR trades gradient efficiency for API-only applicability.
- Computational overhead. Each SPAR iteration requires $O(n_proposals \times n_eval)$ additional API calls beyond the baseline. For large query sets and many proposals, this cost is non-trivial. The Ollama backend eliminates monetary cost but not wall-clock time.
- Convergence. The loop uses a fixed iteration count with a commit threshold δ. There is no formal convergence guarantee. In practice, 3–5 iterations are sufficient for typical RAG workloads, but adversarial or highly out-of-distribution query sets may require more.

© 2025 Ashutosh Sinha. All rights reserved.

18e. Conclusion

This document introduced Sara — a comprehensive knowledge distillation toolkit whose centrepiece is KD-SPAR (Knowledge Distillation via Student Prompt Auto-Rewriting). KD-SPAR operationalises a simple insight: the same model that will execute a system prompt is also best placed to author it, because it has privileged self-knowledge about what instructions elicit its best-aligned responses.

Four variants were developed and fully implemented. The base KD-SPAR loop diagnoses failure modes, conducts a self-interview, aggregates proposals, and commits updates only when KD alignment improves on held-out validation queries. Multi-Teacher KD-SPAR satisfies N specialist teachers simultaneously with non-regression validation. Adversarial KD-SPAR hardness-mines the long tail and applies dual-objective validation to build robustness. Federated KD-SPAR allows distributed client sites to jointly optimise a shared global prompt without sharing any raw query or response data — only instruction strings cross site boundaries, a stronger privacy guarantee than federated gradient sharing.

The critical empirical question — whether student self-authorship adds value beyond external KD-guided proposal — is tested through the controlled four-condition ablation in Section 20. The A–B gap (SARA versus externally-proposed, with identical KD signal) is the metric that reviewers will focus on. Both the Anthropic API and fully local Ollama backends (llama3.1:8b → llama3.2:3b; qwen2.5:7b → llama3.2:3b) are provided so this experiment can be run without cost on a single GPU workstation.

The name 'Sara'

Sara (सार, sāra) in Sanskrit means quintessence — the refined essence extracted from a larger whole. This is exactly what knowledge distillation does: it extracts the sāra of a large teacher model. KD-SPAR adds a second layer: the student finds the sāra of its own failures, and uses that distilled self-understanding to improve. The Federated variant shares only the sāra of insights — instruction strings — never the raw data from which they were derived.

© 2025 Ashutosh Sinha. All rights reserved.

PART VIII

Novelty & Research Analysis

KD-SPAR's Contributions Relative to the Literature

19. Novelties of KD-SPAR

This section offers an honest assessment of KD-SPAR's contributions relative to the existing literature — covering the core novelty claims, a systematic comparison to prior art, and a summary of the research positioning.

19.1 Novelty Assessment

KD-SPAR's core claim is that a language model has privileged self-knowledge about what instructions elicit its best behaviour, and that this self-knowledge can be harnessed to auto-rewrite its own system prompt using KD divergence from a teacher as the optimisation signal. Three properties distinguish KD-SPAR from all existing prompt optimisation methods:

- Self-authorship: the same model that will execute the prompt is the one that authors it. This creates structural self-consistency — the proposed instruction is calibrated to the model's own interpretation of language, not an external model's interpretation of what the model should do.
- KD-grounded objective: the optimisation target is a teacher's output distribution, not human preference, crowdsourced annotations, or task accuracy. This makes KD-SPAR applicable in any domain where a teacher model exists.
- Failure-mode diagnosis: the student first classifies how it failed (citation, calibration, format, completeness) and then proposes targeted fixes for that specific failure type — not generic improvements.

19.2 Comparison to Prior Art

Method	Objective	Model Role	Self-Calibrating?	KD Signal?	API-Only?
OPRO (Yang 2023)	Task accuracy	Black box	No	No	Yes
DSPy (Khattab 2024)	Task metric	Black box	No	Via metric only	Yes
APE (Zhou 2023)	Task accuracy	Generator	No	No	Yes
EvoPrompt (Guo 2023)	Task accuracy	Mutator	No	No	Yes
Constitutional AI	Alignment	Self-critic	Partial	No	No
Self-Refinement	Output quality	Self-editor	Partial	No	Yes

© 2025 Ashutosh Sinha. All rights reserved.

Method	Objective	Model Role	Self-Calibrating?	KD Signal?	API-Only?
Soft Prompt Distil.	KD loss	Gradient	No	Yes	No (needs weights)
KD-SPAR (ours)	KD loss	Self-author	Yes	Yes	Yes
MetaKDSPAR (ours)	KD loss	Multi-specialist self-author	Yes	Yes	Yes

The cell 'Self-Calibrating = Yes' for Constitutional AI and Self-Refinement is marked 'Partial' because these methods improve the model's outputs given a fixed prompt. KD-SPAR improves the model's instructions — the prompt itself — which is structurally different and more durable.

19.3 Summary Assessment

Dimension	Assessment	Confidence
Core novelty	Genuine — self-calibrating KD-driven prompt authorship	High
Prior art risk	Moderate — overlap with Constitutional AI, OPRO, DSPy	Medium
Self-knowledge claim	Testable — A−B gap ablation isolates the mechanism	High
Federated variant	Strong — novel privacy architecture (instruction-only)	High
Multi-teacher variant	Novel — non-regression gate is a new validation protocol	Medium

Author's Assessment

KD-SPAR is a genuine innovation. The intellectual bet — that instruction-tuned models have useful self-knowledge about what instructions elicit their best performance — is clean, testable, and practically motivated. Even a 60% useful proposal rate, combined with robust Phase 4 validation, should produce meaningful improvements over multiple iterations. The Federated variant addresses a real and urgent enterprise problem (multi-site RAG with data sovereignty) with a principled solution. The controlled four-condition ablation in Section 20 directly tests the self-knowledge hypothesis, and the A−B gap is the single number that determines whether the core mechanism claim holds.

© 2025 Ashutosh Sinha. All rights reserved.

20. Experimental Results: Self-Knowledge Hypothesis Test

This section is a placeholder. It is automatically replaced with real experimental results when you run the ablation experiment and execute `patch_paper.py`. See Section 21 for the local model setup and the README for instructions.

To generate this section with real data

bash setup_and_run.sh — or manually: python experiments/kd_spar_ablation_ollama.py --config 1 --iterations 3 --seed 42 | python experiments/collect_results.py | python experiments/patch_paper.py --output docs/paper/Sara_Knowledge_Distillation.pdf

© 2025 Ashutosh Sinha. All rights reserved.

21. Local Model Backend — Running KD-SPAR with Ollama

The Anthropic API provides access to the highest-quality teacher models, but it introduces cost, rate limits, and internet dependency. For rapid experimental iteration, ablation studies, and privacy-sensitive deployments the codebase includes a complete local Ollama backend that runs entirely on your own hardware. No API key, no rate limits, no cost per call.

21.1 Architecture

The Ollama backend is a drop-in replacement for the Anthropic pipeline. All three classes expose the same interface — `.query()`, `.ingest()`, and `.client.update_system()` — so the entire KD-SPAR loop runs unchanged. ChromaDB and sentence-transformers handle retrieval as before; only the generation step moves to the local Ollama server.

Anthropic Class	Ollama Equivalent	Notes
AnthropicClient	OllamaClient	Calls localhost:11434/api/chat
RAGPipeline	OllamaRAGPipeline	Identical interface, local generation
KDSPAR	OllamaKDSPAR	Full 4-phase loop, local models
MultiTeacherKDSPAR	OllamaMultiTeacherKDSPAR	Specialist prompts on same model

21.2 Recommended Model Pairs

Three configurations are provided, covering same-family controlled comparison and cross-family generalisation tests:

Config	Teacher	Student	Disk	Use Case
1 (recommended)	llama3.1:8b	llama3.2:3b	6.7 GB	Same family — controlled capacity gap test
2 (cross-family)	qwen2.5:7b	llama3.2:3b	6.4 GB	Cross-family — tests generalisation of KD signal
3 (Qwen student)	llama3.1:8b	qwen2.5:3b	7.0 GB	Different student arch — architecture-agnostic claim

Config 1 is the recommended starting point: it controls for model family, the 8B→3B capacity ratio matches common production compression targets, and both models are among the best-performing open-weight options at their size. Config 2 is critical for publication — a cross-family A–B gap shows the mechanism generalises beyond a single model provider's training data and tokenisation.

21.3 Local Experimentation Setup — Complete Shell Script

© 2025 Ashutosh Sinha. All rights reserved.

The following is a complete, copy-paste-ready shell script for setting up Sara on a System76 OryxPro running Pop!_OS (Ubuntu-based). Run each block in order. The RTX 3070 Ti accelerates Ollama inference to ~30–50 tokens/sec for 7–8B models, making a full 3-iteration ablation complete in 12–18 minutes. GPU is optional — CPU-only runs take 45–90 minutes but produce identical results.

```
#!/usr/bin/env bash
# Sara (सार) — OryxPro Setup & Ablation Script
# Save as: setup_and_run.sh
# Run with: bash setup_and_run.sh
set -e   # exit on first error
# ── STEP 1: Install Ollama (one-time) ──────────────────────────────
if ! command -v ollama &> /dev/null; then
    curl -fsSL https://ollama.com/install.sh | sh
    echo 'Ollama installed.'
else
    echo 'Ollama already installed, skipping.'
fi
# Start Ollama server in background (auto-starts on Pop!_OS via systemd,
# but start manually if not running)
ollama serve &> /tmp/ollama.log &
sleep 3   # wait for server to be ready
# ── STEP 2: Pull models (one-time, uses disk not GPU) ──────────────
ollama pull llama3.1:8b    # 4.7 GB — recommended teacher  (Config 1 & 3)
ollama pull llama3.2:3b    # 2.0 GB — recommended student  (Config 1 & 2)
ollama pull qwen2.5:7b     # 4.4 GB — cross-family teacher (Config 2)
# ollama pull qwen2.5:3b  # 2.1 GB — optional Qwen student (Config 3)
# ── STEP 3: Project setup ──────────────────────────────────────────
cd ~/PycharmProjects/sara   # adjust path if different
# Create virtual environment (skip if already exists)
if [ ! -d '.venv' ]; then
    python3 -m venv .venv
fi
source .venv/bin/activate
# Install Sara RAG dependencies only (no PyTorch needed for ablation)
pip install -e '.[rag]'
# ── STEP 4: Sanity check ───────────────────────────────────────────
python examples/08_ollama_kd_spar.py --teacher llama3.1:8b --student llama3.2:3b
# ── STEP 5: Run the ablation (publication run: 5 seeds x 2 configs) ──
# Config 1: llama3.1:8b (teacher) -> llama3.2:3b (student) — same family
# Config 2: qwen2.5:7b  (teacher) -> llama3.2:3b (student) — cross family
for cfg in 1 2; do
    for seed in 42 123 456 789 101; do
        echo "Running config=$cfg seed=$seed ..."
        python experiments/kd_spar_ablation_ollama.py \
            --config $cfg --iterations 3 --seed $seed
    done
done
```

© 2025 Ashutosh Sinha. All rights reserved.

```
# ── STEP 6: Aggregate results ──────────────────────────────────────
python experiments/collect_results.py
# ── STEP 7: Patch paper with real numbers ──────────────────────────
# Requires reportlab + pypdf on this machine to rebuild the PDF:
# pip install reportlab pypdf
# python experiments/patch_paper.py --output ~/Desktop/sara_paper.pdf
# ── STEP 8: View results summary ───────────────────────────────────
python experiments/results_analysis.py
echo '================================================'
echo 'Sara ablation complete. Key metric: A-B gap.'
echo 'Results in: experiments/results/'
echo '================================================'
```

Quick run (single config, 3 seeds, ~20 min) for a first result before committing to the full run:

```
source .venv/bin/activate
for seed in 42 123 456; do
    python experiments/kd_spar_ablation_ollama.py --config 1 --iterations 3 --seed $seed
done
python experiments/collect_results.py
python experiments/results_analysis.py
```

Note on pyproject.toml: The project uses an empty base `dependencies = []` list so that `pip install -e '.[rag]'` installs only the four RAG packages (anthropic, chromadb, sentence-transformers, requests) without downloading PyTorch. The `[vision]` and `[nlp]` extras add PyTorch when needed.

21.4 Why Local Models Matter for the Experimentation

Running the ablation on local open-weight models provides three advantages over API-only evaluation:

- Reproducibility: Fixed model weights (same Ollama quantisation) plus temperature=0 gives identical outputs across runs. Reviewers can reproduce your exact numbers by pulling the same model tags.
- Multi-architecture evidence: Showing the A−B gap across llama3.1:8b→llama3.2:3b (same family) and qwen2.5:7b→llama3.2:3b (cross-family) provides much stronger evidence that the self-knowledge mechanism is general rather than specific to one model provider.
- Free iteration: Running 20 ablation seeds costs nothing and takes a few hours overnight. This lets you report tighter confidence intervals and vary the number of SPAR iterations as a hyperparameter sweep.

Further Experimentation Recommendations

Run the ablation on both the Anthropic API pair (claude-3-5-sonnet teacher → claude-sonnet-4-5 student) and the Ollama Config 1 pair (llama3.1:8b → llama3.2:3b) with 3 seeds each. If the A−B gap is positive and consistent across both API and open-weight settings, the claim is robust to model family. That cross-model consistency is the difference between a short paper and a full paper.

© 2025 Ashutosh Sinha. All rights reserved.

References

[1] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. NIPS Deep Learning Workshop. arXiv:1503.02531.

[2] Bucilua, C., Caruana, R., & Niculescu-Mizil, A. (2006). Model Compression. ACM SIGKDD. doi:10.1145/1150402.1150464.

[3] Romero, A. et al. (2015). FitNets: Hints for Thin Deep Nets. ICLR. arXiv:1412.6550.

[4] Park, W. et al. (2019). Relational Knowledge Distillation. CVPR. arXiv:1904.05068.

[5] Zagoruyko, S. & Komodakis, N. (2017). Paying More Attention to Attention. ICLR. arXiv:1612.03928.

[6] Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT. arXiv:1910.01108.

[7] Jiao, X. et al. (2020). TinyBERT: Distilling BERT for NLU. EMNLP. arXiv:1909.10351.

[8] Beyer, L. et al. (2022). Knowledge Distillation: A Good Teacher is Patient and Consistent. CVPR. arXiv:2106.05237.

[9] Microsoft Research (2024). Phi-3 Technical Report. arXiv:2404.14219.

[10] Gou, J. et al. (2021). Knowledge Distillation: A Survey. IJCV 129(6):1789–1819.

[11] Radford, A. et al. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. ICML. arXiv:2212.04356.

[12] Li, X.L. & Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL. arXiv:2101.00190.

[13] Lester, B. et al. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP. arXiv:2104.08691.

[14] Khattab, O. et al. (2024). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR. arXiv:2310.03714.

[15] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS. arXiv:2005.11401.

[16] Yang, C. et al. (2023). Large Language Models as Optimizers (OPRO). arXiv:2309.03409.

[17] Guo, Q. et al. (2023). Connecting Large Language Models with Evolutionary Algorithms (EvoPrompting). arXiv:2309.08532.

[18] Zhou, Y. et al. (2023). Large Language Models are Human-Level Prompt Engineers (APE). ICLR. arXiv:2211.01910.

[19] Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic. arXiv:2212.08073.

[20] Madaan, A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS. arXiv:2303.17651.

[21] Hu, E.J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. ICLR. arXiv:2106.09685.

[22] McMahan, H.B. et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data (FedAvg). AISTATS. arXiv:1602.05629.

[23] Suzgun, M. & Kalai, A.T. (2024). Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding. arXiv:2401.12954.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

Glossary

Term	Definition
Sara (सार)	Sanskrit: quintessence, refined essence extracted from a larger whole. The project name.
Dark Knowledge	Inter-class similarity encoded in teacher soft output distributions.
Temperature (T)	Scalar that flattens the softmax, revealing inter-class structure.
Soft Targets	Teacher output probabilities used as the primary student training signal.
Hard Labels	One-hot ground-truth labels from the original labelled dataset.
Alpha (α)	Mixing coefficient: weight of distillation loss vs. cross-entropy.
Feature Distillation	Matching intermediate hidden-layer activations teacher→student.
Attention Transfer	Distilling spatial attention maps from CNN or transformer layers.
Data-Free KD	Distillation using GAN-synthesised pseudo-data; no original data.
Soft Prompt	Learnable continuous token embeddings prepended to inputs; weights frozen.
KD-SPAR	KD via Student Prompt Auto-Rewriting — student authors its own prompt fixes.
Multi-Teacher KD-SPAR	KD-SPAR variant aligning student to N teachers simultaneously without regression.
Adversarial KD-SPAR	KD-SPAR variant mining hard examples with dual-objective validation.
Federated KD-SPAR	KD-SPAR variant where clients share only instruction strings, not data.
Worst-Teacher Principle	In Multi-Teacher SPAR: target the lowest weighted KD score teacher each iteration.
Dual-Objective Validation	In Adversarial SPAR: candidate must improve hard queries AND not regress on standard.
Gap-Mined Hard Examples	Production queries in the bottom decile of KD scores; real failures already occurring.
Non-Regression Gate	Validation rule: new_score ≥ old_score − tolerance for all secondary teachers.
H_0 (null hypothesis)	E[KD(A)] ≤ E[KD(B)]: self-authorship adds no value beyond the KD signal alone.
H_1 (alternative)	E[KD(A)] > E[KD(B)]: student self-authorship produces higher alignment.
A−B Gap	KD(A) − KD(B): pure value of self-authorship. >0.02 = strong evidence for H_1.
Condition A	SARA self-proposed: student diagnoses and authors its own prompt fixes.
Condition B	Externally proposed: teacher model proposes with the same KD signal.
Condition C	Random instruction baseline: generic pool, no KD signal or diagnosis.
Condition D	No-tuning baseline: student runs on the vanilla default system prompt.

© 2025 Ashutosh Sinha. All rights reserved.

Confidential

Term	Definition
KL Divergence	Information-theoretic distance between two probability distributions.
BERTScore	Semantic similarity metric using contextual BERT embeddings (F1 of alignment).
Jaccard Similarity	Token-overlap ratio; primary KD scoring proxy in this work.
DSPy	Declarative Self-improving Language Programs — LLM pipeline compiler.
OPRO	Optimise by PROmpting — LLM-as-optimiser for task-accuracy objectives.
QAT	Quantisation-Aware Training — simulates quantisation during training.
Self-Knowledge	The model's implicit knowledge of what instructions elicit its best responses.
Ollama	Open-source local LLM runtime; serves models via REST API at localhost:11434.
OllamaKDSPAR	SARA variant using local Ollama models — zero API cost, fully offline.
Llama 3.1 8B	Meta's 8B instruction-tuned model; recommended local SARA teacher.
Llama 3.2 3B	Meta's 3B model; recommended local SARA student (Config 1).
Qwen 2.5 7B	Alibaba's 7B model; cross-family teacher for Config 2 ablation.
OryxPro	System76 OryxPro laptop (Pop!_OS); primary hardware for local experiments.
MetaKDSPAR	KD-SPAR variant using conductor + specialist metaprompting architecture.
Metaprompting	Orchestrating one LLM through multiple specialist personas via a conductor prompt.
Conductor	In MetaKDSPAR: the prompt that synthesises specialist diagnoses and reconciles proposals.
Specialist	In MetaKDSPAR: a domain-specific diagnostic perspective (citation, calibration, etc.).
E\u2212A Gap	KD(E) \u2212 KD(A): value added by multi-perspective diagnosis over flat diagnosis.
Condition E	MetaKDSPAR: conductor + specialist self-diagnosis and proposal.

Typeset with ReportLab Platypus. KD-SPAR contributions by Ashutosh Sinha. ***© 2025 Ashutosh Sinha (ajsinha@gmail.com). All rights reserved.***